VULNERABILITY ASSESSMENT REPORT

Reflected Cross-Site Scripting (XSS) in Search Query Parameter

Mohit Prajapati – Bug Bounty Hunter

Target Application	Henry's Canada (https://www.henrys.com)
Vulnerability Type	CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
Severity Rating	High (CVSS v3.1 Score: 8.2 Overall)
Date of Discovery	October 1, 2026

1. Executive Summary

A high-severity Reflected Cross-Site Scripting (XSS) vulnerability was identified on the official website of Henry's Canada (https://www.henrys.com). The vulnerability exists within the global search feature, where user-supplied input via the query parameters is reflected dynamically back into the application's document object model (DOM) context without adequate contextual output sanitization or encoding. As a result, an attacker could craft a malicious link that executes arbitrary JavaScript inside the victim's browser session.

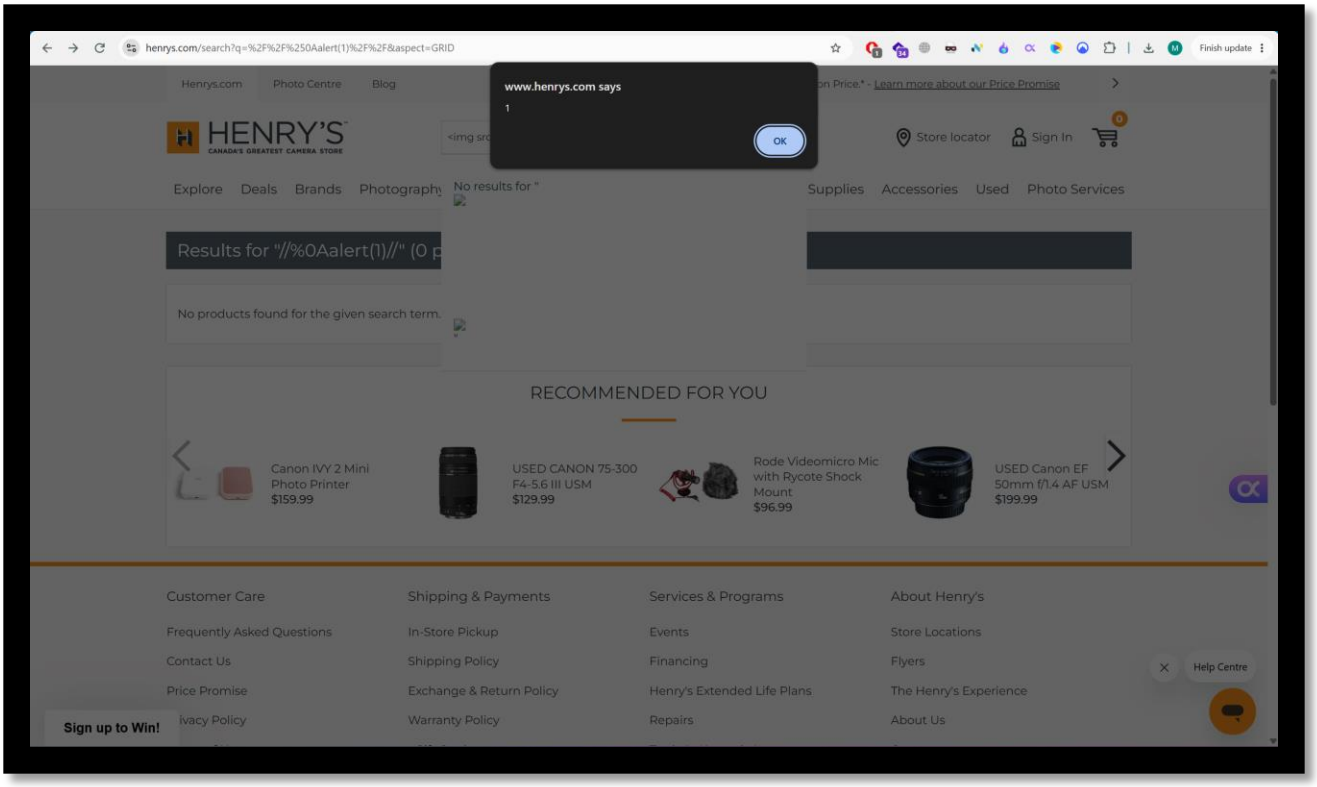
2. Vulnerability Breakdown

• **Vulnerable Parameter:** q (Search query string)
• **Attack Vector:** Reflected / DOM-manipulation script injection
• **Flaw Mechanism:** The application fails to apply strict character entity encoding (such as translating '<' and '>' to safe HTML syntax components) before appending input values directly into structural web layers. This allows traditional HTML tag breakouts and JavaScript execution handles to run transparently.

3. Proof of Concept (PoC)

The bug can be reliably triggered by structuring alternative evasion payloads into the main search parameter. The URL structure demonstrating this execution is outlined below:

```
Vulnerable URL Endpoint Example:
https://www.henrys.com/search?q=%2F%250Aalert(1)%2F&aspect=GRID
```



The visual context provided in user testing shows the browser successfully evaluating payload parameters, culminating in an unconstrained runtime alert execution box demonstrating complete control of the current frame context.

4. Impact Analysis

By forcing a target victim to click a specially crafted URL containing these elements, a remote attacker can execute arbitrary scripting payloads inside that user's session context. This grants full operational access to read local storage parameters, intercept active context tokens, mimic authenticated transactions, or orchestrate highly convincing on-screen credential-harvesting masks.

5. Mitigation & Defenses

5.1 Context-Aware HTML Entity Encoding

Before printing data directly back onto user frames, dynamically convert active syntax triggers into explicit, non-executable textual interpretations:

Character	Safe Entity Mapping	Context
<	<	Prevents element initialization blocks
>	>	Prevents structural bracket completion closes

"	"	Blocks early attribute breakout boundaries
'	'	Secures underlying quote encapsulation paths

5.2 Defensive Platform Control Upgrades

1. Use Secure Text Binding APIs: Utilize framework properties like textContent or innerText instead of directly assigning raw variables to dangerous HTML parsing properties such as innerHTML.

2. Deploy a Content Security Policy (CSP): Implement strong platform restriction headers on server responses to strictly block unauthorized inline scripts from executing natively:
Content-Security-Policy: default-src 'self'; script-src 'self';